University of Bahrain

College of Information Technology
Department of Computer Science

ITCS347 Design and Analysis of Algorithms
Second Semester 2025/2026

Home Assignment #2

Student Name	Abdulnasser Fawzi	GRADE
Student ID	202008866	
Section	3	

Civil Engineering – Resultant Force of Components
When multiple forces act at different angles, the resultant force is:

$$F = \sqrt{\left(\sum F_x\right)^2 + \left(\sum F_y\right)^2}$$

where F_x and F_y are horizontal and vertical components stored in two arrays $Fx[0..n-1]$ and $Fy[0..n-1]$ of real numbers respectively.

1. Design a divide-and-conquer algorithm $\textsc{SumArray}(A, left, right)$ that computes the sum of all elements in a subarray $A[left..right]$.

Solution:
The algorithm divides $A[left..right]$ into two equal halves, recursively computes the sum of each half, and combines the two partial sums by a single addition (the combine step).

Algorithm $\textsc{SumArray}(A,\ left,\ right)$
if $left = right$ **then**
  return $A[left]$
else
  $mid \leftarrow \lfloor (left + right)/2 \rfloor$
  $leftSum \leftarrow \textsc{SumArray}(A,\ left,\ mid)$
  $rightSum \leftarrow \textsc{SumArray}(A,\ mid + 1,\ right)$
  return $leftSum + rightSum$
end if

2. Using $\textsc{SumArray}$, design an algorithm $\textsc{ComputeResult}(Fx, Fy, n)$ that computes the resultant force F.

Solution:

ComputeResult calls SumArray on each component array to obtain $\sum F_x$ and $\sum F_y$, then applies the formula directly.

```
Algorithm ComputeResult(Fx, Fy, n)
sumX ← SumArray(Fx, 0, n − 1)
sumY ← SumArray(Fy, 0, n − 1)
return √(sumX² + sumY²)
```

3. Analyze the algorithm and find the time complexity recurrence relation $T(n)$.

Solution:

- **Problem size:** $n = right - left + 1$ (number of elements in the subarray).
- **Basic operation:** the addition in the combine step (**return** $leftSum+rightSum$).

Analysis:

- **Base case** ($n = 1$, i.e., $left = right$): the algorithm returns $A[left]$ immediately; no addition is performed, so $T(1) = 0$.
- **Recursive case** ($n > 1$): two recursive calls are made, each on a subproblem of size $n/2$, and one addition is performed to combine the results.

The recurrence relation for SumArray is:

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ 2\,T(n/2) + 1 & \text{if } n > 1 \end{cases}$$

4. Prove that $T(n)$ is linear (for the algorithm SumArray use the *Master Theorem*).

Solution:

Applying the Master Theorem to $T(n) = a\,T(n/b) + \Theta(n^d)$, we extract:

$$a = 2, \quad b = 2, \quad d = 0 \qquad \big(\text{since } f(n) = 1 = n^0 \text{ is constant}\big)$$

Compute $b^d = 2^0 = 1$.

Since $a > b^d$ (i.e., $2 > 1$), we apply **Case 3** of the Master Theorem:

$$T(n) \in \Theta\big(n^{\log_b a}\big) = \Theta\big(n^{\log_2 2}\big) = \Theta\big(n^1\big) = \Theta(n)$$

Therefore $T(n) \in \Theta(n)$, which proves that SumArray runs in **linear time**. ■